

Capítulo 5

Seguridad de APIs, autenticación y JWT

Desarrollo de Software IX · Código 1493 · Módulo III

Propósito del capítulo

Presentar las bases teóricas de seguridad para servicios backend:
contraseñas cifradas, autenticación, autorización, tokens JWT, validación,
CORS, secretos y manejo responsable de errores.

Tabla de contenido

1. Panorama del capítulo	3
2. Modelo de amenazas de una API pública	3
3. Autenticación y autorización: dos preguntas distintas	4
4. Teoría del almacenamiento de contraseñas	5
5. Sesiones con estado y tokens sin estado	6
6. Anatomía del JSON Web Token	7
7. Flujo completo: del login al recurso protegido	8
8. Expiración y refresh tokens	9
9. Transporte seguro: HTTPS y el encabezado Authorization	10
10. Validación y saneamiento como defensa	10
11. CORS: por qué existe y qué protege	11
12. Manejo de secretos y configuración	11
13. Caso guía: asegurar la API de reservas	12
14. Actividades sugeridas	12
15. Rúbrica breve	13
16. Cierre	13

1. Panorama del capítulo

La seguridad no es una función extra que se agrega al final. En una aplicación Full Stack, cada capa debe participar: la interfaz orienta al usuario, el backend protege reglas y datos, y la base de datos mantiene información sensible bajo controles adecuados. Este capítulo desarrolla la teoría que sostiene esas decisiones: cómo razona un atacante frente a una API pública, por qué las contraseñas exigen un tratamiento matemático particular, qué problema resuelven los tokens sin estado y qué compromisos aceptamos al adoptarlos.

El recorrido sigue un orden deliberado. Primero se construye un modelo de amenazas: sin entender qué se ataca, las defensas se vuelven rituales sin propósito. Luego se separan con precisión autenticación y autorización, dos conceptos que el lenguaje cotidiano confunde y cuya confusión produce vulnerabilidades reales. Sobre esa base se estudia el almacenamiento de contraseñas, la disyuntiva entre sesiones con estado y tokens sin estado, la anatomía del JSON Web Token, el flujo completo de protección de rutas, y los mecanismos perimetrales: transporte seguro, validación de entradas, CORS y manejo de secretos.

Idea central

El frontend puede mejorar la experiencia, pero el backend debe tomar las decisiones de seguridad. Nunca confiar en que el navegador impedirá acciones maliciosas: todo lo que ejecuta el cliente puede ser leído, alterado o reemplazado por quien lo controla.

2. Modelo de amenazas de una API pública

Un modelo de amenazas es una respuesta ordenada a tres preguntas: qué activos protege el sistema, quién podría querer atacarlos y por qué caminos lo intentaría. Una API expuesta en internet recibe tráfico de cualquier origen, no solo del frontend oficial. Cualquier persona con una terminal puede enviar solicitudes arbitrarias: cambiar identificadores, omitir campos, repetir peticiones miles de veces por segundo o enviar contenido diseñado para confundir al servidor. La superficie de ataque de una API es, exactamente, el conjunto de todas sus rutas con todos sus parámetros posibles, no solo los usos previstos.

Los objetivos típicos de un atacante se agrupan en cuatro familias. Primero, las credenciales: obtener contraseñas por fuerza bruta, por filtración de la base de datos o por reutilización de claves robadas en otros servicios. Segundo, las sesiones e identidades: robar o falsificar el token que representa a un usuario legítimo para actuar en su nombre. Tercero, la inyección: introducir datos que el servidor interprete como instrucciones, alterando consultas o comportamiento interno. Cuarto, la exposición de datos: aprovechar rutas que devuelven más información de la necesaria o que no verifican a quién pertenece el recurso solicitado. El proyecto OWASP organiza estos riesgos en su clasificación API Security Top 10 —no una lista de exploits, sino un mapa conceptual de errores de diseño—, cuyas categorías más relevantes para este curso resume la tabla siguiente.

Riesgo (OWASP API)	Idea esencial	Ejemplo en una API académica
Autorización rota a nivel de objeto (BOLA)	El servidor valida el token pero no verifica que el recurso pertenezca al usuario.	GET /api/reservas/17 devuelve la reserva de otro estudiante.

Riesgo (OWASP API)	Idea esencial	Ejemplo en una API académica
Autenticación rota	Login débil: sin límite de intentos, contraseñas mal almacenadas o tokens mal verificados.	Fuerza bruta sin bloqueo contra /api/auth/login.
Exposición excesiva de datos	La API devuelve el objeto completo y confía en que el frontend filtre.	Respuesta JSON incluye passwordHash y correos ajenos.
Falta de límites de consumo	No hay tope de solicitudes ni de tamaño de carga.	Un script descarga toda la base con miles de peticiones.
Asignación masiva	El servidor copia todos los campos recibidos al modelo.	El cliente envía rol: 'admin' al registrarse y el backend lo acepta.
Configuración insegura	CORS permisivo, mensajes de error con detalles internos, secretos expuestos.	Stack trace completo devuelto en producción.
Inyección	Datos del cliente interpretados como instrucciones.	Operadores especiales dentro del cuerpo JSON alteran una consulta.

Perspectiva

La pregunta de diseño no es si alguien intentará abusar de la API, sino qué ocurrirá cuando lo intente. Cada ruta debe pensarse dos veces: una desde el uso legítimo y otra desde el abuso deliberado.

3. Autenticación y autorización: dos preguntas distintas

Autenticación significa comprobar quién es el usuario. Autorización significa comprobar qué puede hacer ese usuario. Son conceptos relacionados, pero no equivalentes, y el orden importa: primero se establece identidad y solo después tiene sentido evaluar permisos. Un usuario puede estar perfectamente autenticado y aun así no tener permiso para modificar un recurso. Confundir ambas preguntas conduce al error más común en APIs estudiantiles: verificar que existe un token válido y asumir que eso basta para autorizar cualquier operación. La autorización, además, tiene dos niveles que deben evaluarse por separado. El nivel de rol responde a preguntas globales: ¿puede un estudiante aprobar reservas? El nivel de objeto responde a preguntas particulares: este estudiante autenticado, ¿es dueño de la reserva 17 que intenta modificar? Un sistema puede tener roles impecables y aun así sufrir autorización rota a nivel de objeto, porque nunca compara el identificador del recurso con el identificador del solicitante. Por eso la autorización debe verificarse en cada solicitud y contra cada recurso, no una sola vez al iniciar sesión.

Concepto	Pregunta	Ejemplo
Autenticación	¿Quién eres?	Login con correo y contraseña.
Autorización	¿Qué puedes hacer?	Solo administrador aprueba solicitudes.
Sesión/token	¿Cómo se recuerda tu identidad?	JWT enviado en cada solicitud.
Rol/permiso	¿Qué reglas aplican?	estudiante, docente, administrador.
Propiedad	¿Es tuyo este recurso?	Solo el autor edita su propia reserva.

4. Teoría del almacenamiento de contraseñas

Las contraseñas nunca deben guardarse en texto plano, y la razón es estructural, no cosmética: las bases de datos se filtran. Por error de configuración, por respaldo extraviado, por un empleado descontento o por un ataque exitoso, debe asumirse que tarde o temprano un tercero leerá la tabla de usuarios. Si las contraseñas están en claro, el daño trasciende al sistema propio, porque las personas reutilizan claves: la contraseña filtrada de una aplicación académica suele abrir también su correo personal o su banca. Almacenar texto plano convierte cada brecha local en una brecha global para los usuarios. La segunda tentación, el cifrado reversible, también es incorrecta, aunque por una razón más sutil: cifrar implica que existe una clave capaz de descifrar; esa clave debe vivir en algún lugar accesible para el servidor, y quien comprometa el servidor obtiene la clave junto con los datos. Pero el argumento decisivo es conceptual: el sistema no necesita recuperar la contraseña original jamás. Solo necesita responder una pregunta en el login: ¿la contraseña que acaba de llegar es la misma que se registró? Para responder eso basta una función unidireccional.

Esa función es el hash criptográfico: una transformación que produce una huella de tamaño fijo, fácil de calcular hacia adelante y computacionalmente impracticable de invertir. Al registrarse, el sistema guarda hash(contraseña); al iniciar sesión, calcula el hash de lo recibido y compara huellas. Hashing no es lo mismo que cifrado reversible: no hay clave, no hay vuelta atrás, y eso es precisamente la virtud. Sin embargo, un hash genérico y rápido como SHA-256 no basta para contraseñas, porque la velocidad favorece al atacante.

Contra una tabla de hashes robada operan dos estrategias clásicas. El ataque de diccionario precalcula los hashes de millones de contraseñas comunes y busca coincidencias; las tablas arcoíris industrializan esa idea. La defensa es la sal: un valor aleatorio único por usuario que se concatena a la contraseña antes de aplicar el hash. Con sal, dos usuarios con la misma contraseña producen hashes distintos, y todo precálculo se vuelve inútil, porque el atacante tendría que repetir el trabajo para cada sal individual. La sal no es secreta; se guarda junto al hash, y su poder reside en la unicidad, no en la ocultación.

La segunda estrategia es la fuerza bruta directa: probar combinaciones contra cada hash robado. Aquí la defensa es el factor de costo. Algoritmos diseñados para contraseñas, como bcrypt, son deliberadamente lentos y ajustables: el parámetro de costo indica cuántas rondas internas ejecutar, y cada incremento duplica el trabajo. Para un usuario legítimo, cien milisegundos en el login son imperceptibles; para un atacante que necesita probar miles de millones de candidatos, esa lentitud multiplica el ataque de horas a siglos. bcrypt, además, genera e incrusta la sal automáticamente dentro del propio hash, de modo que la comparación posterior no requiere gestionarla aparte.

Estrategia	¿Recuperable?	¿Resiste filtración?	Veredicto
Texto plano	Sí (trivialmente)	No: exposición total e inmediata.	Inaceptable.
Cifrado reversible	Sí (con la clave)	No: la clave cae con el servidor.	Inaceptable para contraseñas.
Hash rápido (SHA-256)	No	Débil: diccionarios y GPU lo recorren a gran velocidad.	Insuficiente.
Hash lento con sal (bcrypt)	No	Sí: sal única más costo ajustable frenan el ataque masivo.	Práctica esperada.

```
// Registro: derivar y guardar el hash, nunca la contraseña.
const bcrypt = require('bcrypt');
const passwordHash = await bcrypt.hash(password, 10); // 10 = factor de costo
await User.create({ email, passwordHash });

// Login: comparar sin descifrar nada.
const user = await User.findOne({ email });
const ok = user && await bcrypt.compare(password, user.passwordHash);
if (!ok) return res.status(401).json({ error: 'Credenciales inválidas' });
```

- Guardar `passwordHash`, no `password`.
- Aplicar una política mínima de longitud y complejidad.
- No devolver hashes en respuestas JSON ni incluirlos al serializar el modelo.
- No registrar contraseñas en consola o logs.
- Usar mensajes de error que no revelen si existe un correo: «credenciales inválidas» protege más que «contraseña incorrecta».

5. Sesiones con estado y tokens sin estado

Una vez autenticado el usuario, el sistema necesita recordarlo, porque HTTP no guarda memoria entre solicitudes. La solución clásica es la sesión con estado: tras el login, el servidor crea un registro de sesión en su memoria o base de datos, y entrega al navegador una cookie con un identificador opaco. En cada solicitud posterior, el servidor busca ese identificador en su almacén y recupera quién es el usuario. La identidad vive en el servidor; el cliente solo porta una llave sin significado propio. El enfoque alternativo invierte la responsabilidad: el token sin estado. El servidor emite, una sola vez, un documento firmado que contiene la identidad y sus atributos; el cliente lo presenta en cada solicitud, y el servidor lo verifica matemáticamente sin consultar ningún almacén. No hay registro de sesión que mantener ni sincronizar. Esta propiedad se vuelve decisiva con el escalado horizontal: cuando la aplicación corre en varias instancias detrás de un balanceador, las sesiones con estado obligan a compartir el almacén entre todas las instancias o a fijar cada usuario a una máquina; con tokens, cualquier instancia que conozca el secreto de firma puede atender a cualquier usuario, sin coordinación alguna.

Criterio	Sesión con estado	Token sin estado (JWT)
Dónde vive la identidad	En el servidor; el cliente porta solo un identificador opaco.	En el propio token, firmado por el servidor.
Costo por solicitud	Búsqueda en memoria o base de datos.	Verificación criptográfica de la firma.
Revocación inmediata	Trivial: borrar la sesión del almacén.	Difícil: el token sigue siendo válido hasta expirar.
Escalado horizontal	Requiere almacén compartido o afinidad de sesión.	Natural: cualquier instancia verifica con el secreto.
Cierre de sesión real	Efectivo en el servidor.	Solo elimina la copia del cliente; la firma sigue válida.

El costo oculto

La revocación es el precio del token sin estado. Si un JWT es robado o un usuario es suspendido, el servidor no puede «apagar» ese token: nada en su verificación consulta el estado actual. Las mitigaciones —expiraciones cortas, listas de bloqueo, rotación de secretos— reintroducen, en pequeñas dosis, el estado que se quiso eliminar. Elegir tokens es aceptar conscientemente ese intercambio.

6. Anatomía del JSON Web Token

JWT permite representar datos de identidad y autorización en un token firmado. El backend lo emite cuando el usuario inicia sesión; el cliente lo envía en solicitudes posteriores, normalmente en el encabezado `Authorization`, y el backend verifica firma, expiración y contenido antes de permitir acceso. Físicamente, un JWT es una cadena con tres segmentos separados por puntos: header, payload y signature. Los dos primeros son JSON codificado en Base64URL, una codificación de transporte, no un cifrado: cualquiera que posea el token puede decodificarlos y leerlos.

```
eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9
eyJzdWIiOiI2NjJhMTciLCJyb2wiOiJlc3R1ZGlhbnRlIiw
Im1ldCI6MTc0OTc0MjAwMCwiZXhwIjoxNzQ5NzQ1NjAwfQ
.4kx7Rz0eYq3T9pXnVtBwLmCaUg8HsdJfKei2NwQ5vM0
<- header (Base64URL)
<- payload (Base64URL)
<- signature

// Header decodificado: { "alg": "HS256", "typ": "JWT" }
// Payload decodificado: { "sub": "662a17", "rol": "estudiante",
//                          "iat": 1749742000, "exp": 1749745600 }
```

Parte	Contenido	Consideración
Header	Algoritmo y tipo de token.	No contiene datos sensibles.
Payload	Identificador, rol, expiración u otros claims.	No guardar secretos; puede leerse si se intercepta.
Signature	Firma con secreto del servidor.	Protege integridad, no confidencialidad.

El payload se compone de claims: afirmaciones sobre el sujeto del token. El estándar reserva nombres con semántica acordada, y conviene usarlos en lugar de inventar equivalentes, porque las librerías los validan automáticamente. Los tres esenciales son `sub` (subject, el identificador del usuario), `iat` (issued at, momento de emisión) y `exp` (expiration, momento a partir del cual el token deja de ser válido). Una librería sería rechaza por sí sola un token cuyo `exp` ya pasó; esa validación automática es parte del valor del estándar.

Claim	Nombre completo	Significado
sub	Subject	Identificador del usuario o entidad autenticada.
iat	Issued at	Marca de tiempo de emisión del token.
exp	Expiration	Momento en que el token deja de ser válido.
iss / aud	Issuer / Audience	Quién emitió el token y para qué sistema está destinado.

La firma admite dos familias de algoritmos con implicaciones de arquitectura distintas. HMAC (por ejemplo HS256) usa criptografía simétrica: el mismo secreto firma y verifica, lo que es simple y suficiente cuando un único backend emite y consume sus propios tokens. RSA y similares (RS256) usan criptografía asimétrica:

una clave privada firma y una clave pública verifica, lo que permite que servicios independientes validen tokens sin poseer jamás el material capaz de emitirlos. La regla práctica: HMAC para un backend monolítico como el del curso; firma asimétrica cuando múltiples servicios deben verificar sin poder firmar.

Regla práctica

JWT firmado no significa información oculta. La firma garantiza que el contenido no fue alterado y que lo emitió quien posee el secreto; no impide leerlo. No colocar contraseñas, documentos sensibles o datos privados innecesarios dentro del payload: todo lo que viaja en él es público para quien tenga el token.

7. Flujo completo: del login al recurso protegido

La teoría anterior se materializa en un ciclo que toda aplicación con JWT repite. Conviene estudiarlo como flujo, porque cada paso responde a un riesgo identificado en el modelo de amenazas: el hash protege la credencial almacenada, la firma protege la identidad en tránsito lógico, el middleware centraliza la verificación y la autorización por rol o propiedad cierra el acceso a nivel de objeto.

- 1 El cliente envía correo y contraseña a POST /api/auth/login.
- 2 El backend localiza al usuario y compara la contraseña recibida contra el hash almacenado con bcrypt.
- 3 Si coincide, firma un JWT con sub, rol y exp, y lo devuelve al cliente.
- 4 El cliente conserva el token y lo adjunta en el encabezado Authorization de cada solicitud protegida.
- 5 Un middleware verifica firma y expiración antes de que la solicitud alcance la lógica de negocio; si el token falla, responde 401 sin ejecutar nada más.
- 6 El controlador aplica la autorización fina: rol requerido o propiedad del recurso; si falta permiso, responde 403.

```
// auth.controller.js – emisión del token tras validar credenciales
const jwt = require('jsonwebtoken');

async function login(req, res) {
  const { email, password } = req.body;
  const user = await User.findOne({ email });
  const ok = user && await bcrypt.compare(password, user.passwordHash);
  if (!ok) return res.status(401).json({ error: 'Credenciales inválidas' });

  const token = jwt.sign(
    { sub: user.id, rol: user.rol },
    process.env.JWT_SECRET,
    { expiresIn: '1h' }
  );
  return res.json({ token });
}
```



```
// auth.middleware.js – verificación centralizada y autorización por rol
function requireAuth(req, res, next) {
  const header = req.headers.authorization || '';
  const token = header.startsWith('Bearer ') ? header.slice(7) : null;
  if (!token) return res.status(401).json({ error: 'Token requerido' });
  try {
    req.user = jwt.verify(token, process.env.JWT_SECRET); // firma + exp
    next();
  } catch {
    return res.status(401).json({ error: 'Token inválido o expirado' });
  }
}

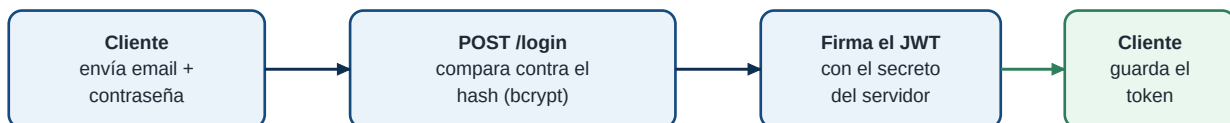
const requireRole = (rol) => (req, res, next) =>
  req.user.rol === rol ? next() : res.status(403).json({ error: 'Sin permiso' });

// Rutas: pública, privada y restringida por rol
app.get('/api/labs', listarLabs);
app.get('/api/me', requireAuth, verPerfil);
app.get('/api/admin/users', requireAuth, requireRole('admin'), listarUsuarios);
```

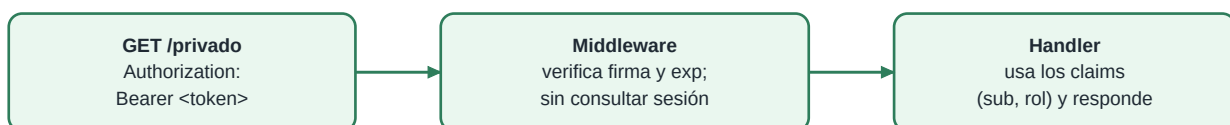
Obsérvese la distinción de códigos: 401 significa «no sé quién eres» (token ausente, inválido o expirado) y 403 significa «sé quién eres, pero no puedes hacer esto». Una ruta privada debe validar token antes de ejecutar la lógica principal; una ruta con permisos debe validar además el rol o la propiedad del recurso. Por ejemplo, un estudiante puede ver sus solicitudes, pero no necesariamente las de todos: verificar solo el token y omitir la comparación de propiedad es, exactamente, la autorización rota a nivel de objeto descrita en el modelo de amenazas.

Ruta	Control mínimo	Riesgo si falta
GET /api/me	Token válido.	Acceso anónimo a datos de usuario.
GET /api/admin/users	Token y rol admin.	Exposición de usuarios.
PUT /api/solicitudes/:id	Token y propiedad o rol.	Modificación de datos ajenos.
DELETE /api/recurso/:id	Token, rol y confirmación de regla.	Eliminación indebida.

1) Autenticación (una vez)



2) Acceso (en cada petición)



firma inválida o token vencido → 401 sin tocar el handler

8. Expiración y refresh tokens

Si los tokens sin estado no pueden revocarse, la expiración es la única frontera garantizada de un token comprometido. De ahí la tensión central: un token de vida larga es cómodo —el usuario casi nunca vuelve a iniciar sesión— pero amplía la ventana de abuso si es robado; un token de vida corta reduce esa ventana, pero expulsa al usuario constantemente. El claim exp convierte esta decisión de riesgo en un parámetro explícito del diseño.

El patrón de refresh tokens resuelve la tensión separando responsabilidades en dos credenciales. El access token es de vida corta —minutos— y viaja con cada solicitud; es el que se expone más y el que menos vale robar. El refresh token es de vida larga, se guarda con más cuidado, se usa únicamente contra una ruta dedicada de renovación y, a diferencia del access token, el servidor sí lo registra: puede invalidarlo individualmente. Cuando el access token expira, el cliente presenta el refresh token y recibe un access token nuevo, sin pedir credenciales otra vez. La rotación añade una defensa más: cada uso del refresh token lo consume y emite uno distinto, de modo que un refresh token robado y reutilizado delata la intrusión. Conceptualmente, el patrón reconoce que algo de estado es inevitable, y lo concentra donde más rinde.

Aspecto	Access token	Refresh token
Vida útil	Corta: minutos a una hora.	Larga: días o semanas.
Frecuencia de uso	En cada solicitud protegida.	Solo al renovar el access token.
Estado en el servidor	Ninguno: verificación pura de firma.	Registrado: puede revocarse individualmente.
Impacto si es robado	Limitado por la expiración corta.	Alto; se mitiga con rotación y detección de reúso.

9. Transporte seguro: HTTPS y el encabezado Authorization

Toda la criptografía anterior se anula si el token viaja por un canal observable. Un JWT interceptado es una identidad robada lista para usar: quien lo posee es el usuario, hasta que expire. HTTP plano expone cada solicitud a cualquier intermediario de la red; HTTPS (TLS) cifra el canal completo, de modo que encabezados y cuerpos resultan ilegibles en tránsito, y además autentica al servidor, impidiendo que un impostor se interponga. Por convención, el token se envía en el encabezado Authorization con el esquema Bearer —literalmente, «portador»: el servidor atiende a quien porte el token, lo que subraya por qué protegerlo en tránsito y en reposo es crítico. Debe evitarse colocar tokens en la URL: las URLs quedan en historiales, logs de servidores y encabezados de referencia.

```
GET /api/me HTTP/1.1
Host: api.ejemplo.edu.pa
Authorization: Bearer eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9.eyJzdWIi...
```

```
// En el cliente (fetch):
fetch('/api/me', { headers: { Authorization: `Bearer ${token}` } });
```

10. Validación y saneamiento como defensa

Validar es comprobar que un dato cumple reglas esperadas. Sanear es limpiar o normalizar datos para reducir riesgos. El backend debe validar siempre, incluso si el frontend ya lo hizo: la validación del frontend mejora usabilidad; la del backend protege el sistema, porque es la única que un atacante no puede

desactivar. La validación es, además, la defensa primaria contra la inyección y la asignación masiva del modelo de amenazas: un servidor que solo acepta los campos que espera, con los tipos y formas que espera, le niega al atacante el vehículo para introducir instrucciones u otorgarse atributos.

- Validar tipos, longitudes, campos requeridos y formatos.
- Rechazar campos no permitidos si pueden modificar comportamiento interno: construir el objeto explícitamente, nunca copiar `req.body` completo al modelo.
- Normalizar correos o identificadores cuando aplique.
- Aplicar límites a texto largo y archivos.
- Responder con errores claros pero sin filtrar detalles internos: el mensaje orienta al cliente legítimo sin entregar un mapa del sistema al atacante.

11. CORS: por qué existe y qué protege

Los navegadores aplican la política del mismo origen: por defecto, el código JavaScript de una página solo puede leer respuestas de su propio origen (combinación de protocolo, dominio y puerto). Esta regla impide que una página maliciosa abierta en el navegador del usuario lea, con las credenciales de ese usuario, datos de otros sitios donde tiene sesión. CORS (Cross-Origin Resource Sharing) es el mecanismo por el cual un servidor relaja esa restricción de forma controlada, declarando mediante encabezados qué orígenes ajenos pueden leer sus respuestas. Cuando el frontend corre en un origen distinto al de la API —situación normal en desarrollo Full Stack—, la API debe autorizar explícitamente ese origen. Es esencial entender, a la vez, qué no hace CORS: no autentica, no autoriza y no detiene a un atacante con una terminal, porque solo los navegadores lo aplican; cualquier cliente directo ignora esos encabezados por completo. CORS protege a los usuarios del navegador contra sitios maliciosos que intenten leer datos ajenos a través de su sesión; no protege a la API de solicitudes hostiles. Por eso no reemplaza autenticación, y por eso configurar el comodín `*` sin reflexión no «abre un agujero de login», pero sí renuncia a una capa de defensa para los usuarios sin que medie una decisión consciente.

Elemento	Buena práctica	Error común
Origen permitido	Permitir solo el origen del frontend conocido.	Usar <code>*</code> sin entender consecuencias.
Credenciales	Habilitar <code>credentials</code> solo si de verdad se usan cookies.	Activarlo «por si acaso» junto a orígenes amplios.
Rol de CORS	Tratarlo como protección del usuario en el navegador.	Creer que sustituye autenticación o autorización.

12. Manejo de secretos y configuración

El secreto de firma JWT, las cadenas de conexión y las claves de servicios externos comparten una propiedad: quien los obtiene, obtiene el sistema. Con el `JWT_SECRET` un atacante no necesita robar tokens; puede fabricarlos, con cualquier identidad y cualquier rol. Por eso los secretos no pertenecen al código fuente: el código se comparte, se clona, se sube a repositorios y se revisa en pantallas; el secreto debe vivir en variables de entorno, inyectadas por el sistema donde corre la aplicación. El repositorio incluye un `.env.example` con los nombres de las variables y valores ficticios —documenta el contrato de

configuración—, mientras el `.env` real queda excluido mediante `.gitignore` y nunca se publica. Los secretos, además, tienen ciclo de vida: la rotación —reemplazarlos periódicamente o ante cualquier sospecha— limita cuánto tiempo vale un secreto robado. Rotar el `JWT_SECRET` invalida de golpe todos los tokens emitidos, lo cual es a la vez un costo (todos los usuarios deben reautenticarse) y la herramienta de revocación masiva más contundente disponible en un esquema sin estado.

```
# .env.example – se versiona; documenta qué variables existen
PORT=3000
MONGODB_URI=mongodb://localhost:27017/dsix_reservas
JWT_SECRET=cambiar-por-un-valor-largo-y-aleatorio
JWT_EXPIRES_IN=1h

# .env – real, NUNCA se versiona (listado en .gitignore)
```

Regla sin excepciones

Un secreto que tocó un repositorio está comprometido, aunque el commit se borre después: el historial de Git lo conserva y los rastreadores automáticos de repositorios públicos lo encuentran en minutos. La única respuesta correcta es rotarlo de inmediato; eliminar el archivo no deshace la exposición.

13. Caso guía: asegurar la API de reservas

El caso guía del curso —la API de reservas de laboratorios— permite aplicar el capítulo completo como una matriz de decisiones. Para cada ruta se identifica la amenaza dominante y la defensa que este capítulo fundamenta. El ejercicio valioso no es memorizar la tabla, sino poder reconstruirla: dada cualquier ruta nueva, derivar amenaza y defensa desde el modelo de la sección 2.

Ruta	Amenaza dominante	Defensa aplicada
POST /api/auth/login	Fuerza bruta de credenciales; enumeración de correos.	bcrypt con factor de costo; mensaje uniforme de credenciales inválidas; límite de intentos.
GET /api/labs	Consumo abusivo (scraping, sobrecarga).	Ruta pública con límites de tasa y paginación.
POST /api/reservas	Asignación masiva; datos inválidos.	Token válido; construcción explícita del objeto; validación de campos y formatos.
GET /api/reservas/:id	Autorización rota a nivel de objeto (BOLA).	Token válido y comparación de propiedad: sub del token contra dueño del recurso.
PUT /api/reservas/:id/estado	Escalada de privilegios (estudiante aprueba reservas).	Token válido y rol docente o administrador; 403 si el rol no alcanza.

14. Actividades sugeridas

- Dibujar el flujo de login de su proyecto, marcando dónde se hashea, dónde se firma y dónde se verifica.
- Construir un modelo de amenazas mínimo del proyecto: activos, atacantes plausibles y rutas de ataque, usando la tabla OWASP como guía.
- Definir roles y permisos mínimos, distinguiendo autorización por rol de autorización por propiedad.

- Indicar qué rutas serán públicas, privadas o por rol, y qué claim del token decide cada caso.
- Crear una matriz de amenazas básicas: acceso indebido, datos inválidos, secretos expuestos.
- Probar una ruta privada sin token, con token inválido, con token expirado y con token válido, verificando 401 y 403 según corresponda.
- Decodificar el payload de un JWT propio (sin verificarlo) y constatar que es legible: justificar por escrito qué datos no deben ir allí.
- Verificar que ``.env`` está en ``.gitignore``, que ``.env.example`` documenta todas las variables y que ningún secreto aparece en el historial del repositorio.

15. Rúbrica breve

Criterio	Excelente	Satisfactorio	Insuficiente
Contraseñas	Hash seguro y no exposición en respuestas/logs.	Hash aplicado con detalles menores.	Texto plano o manejo inseguro.
JWT	Token con expiración, validación y middleware claro.	Token funcional con omisiones.	Token mal usado o sin protección real.
Autorización	Roles/permisos conectados al dominio y verificación de propiedad.	Controles básicos.	Todo usuario puede hacer todo.
Configuración	Secretos en entorno y <code>`.env.example`</code> correcto.	Algunas variables documentadas.	Secretos en repositorio o sin documentar.

16. Cierre

La seguridad mínima esperada en el curso no busca reemplazar una auditoría profesional, pero sí formar criterio. Ese criterio tiene forma de razonamiento, no de lista: partir de las amenazas, separar identidad de permiso, almacenar contraseñas de modo que su filtración no sea catastrófica, entender qué garantiza y qué no garantiza una firma, y tratar los secretos como lo que son. Proteger contraseñas, validar datos, controlar rutas y evitar exposición accidental de secretos son las manifestaciones visibles de un hábito más profundo: diseñar cada ruta pensando, también, en quien intentará abusar de ella.